

Assignment -4

```
import java.io.*;
import java.util.*;
import java.util.spliterator.*;
import java.util.stream.*;
```

```
class Book implements Comparable<Book> {
    private int bookId;
    private String title, author, category;
    private boolean isIssued;
```

```
    Book(int id, String t, String a, String c, boolean issued) {
        this.bookId = id; this.title = t; this.author = a; this.category = c;
        this.isIssued = issued;
    }
```

```
    int getBookId() { return bookId; }
    String getTitle() { return title; }
    String getAuthor() { return author; }
    String getCategory() { return category; }
    boolean isIssued() { return isIssued; }
```

```
    void displayBookDetails() {
        System.out.printf("ID:%d | %s | Category: %s | \n",
            bookId, title, author, category,
            isIssued ? "Yes" : "No");
    }
```

```
    void markAsIssued() { isIssued = true; }
    void markAsReturned() { isIssued = false; }
```

```
String toLine() {
    return bookId + "|" + esc(title) + "|" + esc(author) +
        "|" + esc(category) + "|" + isIssued;
}
```

```
static Book fromLine(String line) {
    String[] p = line.split("\\|", -1);
    int id = Integer.parseInt(p[0]);
    return new Book(id, unesc(p[1]), unesc(p[2]),
        unesc(p[3]),
        Boolean.parseBoolean(p[4]));
}
```

```
private static String esc(String s) { return s.replace("\"", "\\"); }
private static String unesc(String s) { return s.replace("\\\"", ""); }
```

@Override

```
public int compareTo(Book o) {
    return this.title.compareToIgnoreCase(o.title);
}
```

}

class Member {

```
    private int memberId;
    private String name, email;
    private List<Integer> issuedBooks;
```

```
    Member(int id, String name, String email, List<Integer> issued) {
        this.memberId = id; this.name = name; this.email = email;
        this.issuedBooks = issued == null ? new ArrayList<>(): issued;
    }
```

```
int getMemberId() { return memberId; }
String getName() { return name; }
String getEmail() { return email; }
List<Integer> getIssuedBooks() { return issuedBooks; }
```

```
void displayMemberDetails() {
    String sb = issuedBooks.isEmpty() ? "None" : issuedBooks.
    stream().map(Obj::toString).collect(Collectors.joining(", "));
    System.out.printf("ID: %d | %s | %s | IssuedBooks: %s\n",
        memberId, name, email, sb);
}
```

```
void addIssuedBook(int bookId) {
    if (!issuedBooks.contains(bookId)) issuedBooks.add(bookId);
}
```

```
String toLine() {
    String s = issuedBooks.isEmpty() ? "" : issuedBooks.stream().
    map(Obj::toString).collect(Collectors.joining(", "));
    return memberId + "|" + name + "|" + email + "|" + s;
}
```

```
static Member fromLine(String line) {
    String[] p = line.split("\\|", -1);
    int id = Integer.parseInt(p[0]);
    String name = unesc(p[1]);
    String email = unesc(p[2]);
    List<Integer> issued = new ArrayList<>();
    if (p.length > 3 && !p[3].trim().isEmpty()) {
        for (String s : p[3].split(",")) issued.add(Integer.parseInt(s));
    }
    return new Member(id, name, email, issued);
}
```

```
private static String ex(String s) { return s.replace("/", ""); }  
private static String unex(String s) { return s.replace("/", "/"); }
```

```
}  
class LibraryManager {
```

```
    Map<Integer, Book> books = new HashMap<>();
```

```
    Map<Integer, Member> members = new HashMap<>();
```

```
    Set<String> categories = new HashSet<>();
```

```
    private File booksFile = new File("books.txt");
```

```
    private File membersFile = new File("members.txt");
```

```
    private int nextBookId = 100;
```

```
    private int nextMemberId = 200;
```

```
    private Pattern emailPattern = Pattern.compile("[\\w.-]+@[\\w.-]+\\.([A-Za-z]{2,63})$");
```

```
    LibraryManager() {
```

```
        ensureFilesExist();
```

```
        loadFiles();
```

```
    }
```

```
    private void ensureFilesExist() {
```

```
        try {
```

```
            if (!booksFile.exists()) booksFile.createNewFile();
```

```
            if (!membersFile.exists()) membersFile.createNewFile();
```

```
        } catch (IOException e) {
```

```
            System.out.println("Error creating data files: " +
```

```
                e.getMessage());
```

```
        }
```

```
    }
```

```
    // Add book
```

```
    Book addBook(String title, String author, String category) {
```

```
        Book b = new Book(nextBookId++, title.trim(), author.  
            trim(), category.trim(), false);
```

```

books.put(b.getBookId(), b);
categories.add(category.trim());
saveBooks();
return b;
}

```

```

Member addMember(String name, String email) {
    if (!EmailPattern.matcher(email).matches()) throw new
        IllegalArgumentException("Invalid email");
    Member m = new Member(nextMemberId++, name.trim(),
        email.trim(), null);
    members.put(m.getMemberId(), m);
    saveMembers();
    return m;
}

```

```

String IssueBook(int bookId, int memberId) {
    Book b = books.get(bookId);
    Member m = members.get(memberId);
    if (b == null) return "Book not found";
    if (m == null) return "Member not found";
    if (b.isIssued()) return "Book already issued";
    b.markAsIssued();
    m.addIssuedBook(bookId);
    saveBooks();
    saveMembers();
    return "Book issued successfully";
}

```

```

String returnBook(int bookId, int memberId) {
    Book b = books.get(bookId);
    Member m = members.get(memberId);
    if (b == null) return "Book not found";
    if (m == null) return "Member not found";
    if (!b.isIssued()) return "That book is not issued";
}

```

```

        b.markAsReturned();
        m.setReturnedBook(bookId);
        saveBooks();
        saveMembers();
        return "Book returned successfully";
    }

```

```

    List<Book> searchBooks(String q) {
        String s = q.toLowerCase();
        List<Book> res = new ArrayList<>();
        for (Book b : books.values()) {
            if (b.getTitle().toLowerCase().contains(s) ||
                b.getAuthor().toLowerCase().contains(s) ||
                b.getCategory().toLowerCase().contains(s)) res.add(b);
        }
        return res;
    }

```

```

    List<Book> sortByTitle() {
        List<Book> list = new ArrayList<>(books.values());
        Collections.sort(list);
        return list;
    }

```

```

    List<Book> sortByAuthor() {
        List<Book> list = new ArrayList<>(books.values());
        list.sort(Comparator.comparing(Book::getAuthor, String.CASE_INSENSITIVE_ORDER));
        return list;
    }

```

```

    List<Book> sortByCategory() {
        List<Book> list = new ArrayList<>(books.values());
        list.sort(Comparator.comparing(Book::getCategory, String.CASE_INSENSITIVE_ORDER));
        return list;
    }

```

```

    Optional<Book> findBook(int id) { return Optional.ofNullable(books.get(id)); }

```

```

    Optional<Member> findMember(int id) { return Optional.ofNullable(members.get(id)); }

```

```

    Collection<Book> allBooks() { return books.values(); }
    Collection<Member> allMembers() { return members.values(); }

```

```
private void loadFiles() {
```

```
try (BufferedReader br = new BufferedReader(new FileReader(
    booksFile))) {
```

```
String line;
```

```
while ((line = br.readLine()) != null) {
```

```
if (line.trim().isEmpty()) continue;
```

```
Book b = Book.fromLine(line);
```

```
books.put(b.getBookId(), b);
```

```
categories.add(b.category());
```

```
nextBookId = Math.max(nextBookId, b.getBookId() + 1);
```

```
}
```

```
} catch (IOException e) {
```

```
System.out.println("Error reading books file: " + e.getMessage());
```

```
e.getMessage();
```

```
}
```

```
try (BufferedReader br = new BufferedReader(new FileReader(
    membersFile))) {
```

```
String line;
```

```
while ((line = br.readLine()) != null) {
```

```
if (line.trim().isEmpty()) continue;
```

```
Member m = Member.fromLine(line);
```

```
members.put(m.getMemberId(), m);
```

```
nextMemberId = Math.max(nextMemberId,
```

```
m.getMemberId() + 1);
```

```
}
```

```
} catch (IOException e) {
```

```
System.out.println("Error reading members file: " +
```

```
e.getMessage());
```

```
}
```

```
}
```

```
private void saveBooks() {
```

```
try (BufferedWriter bw = new BufferedWriter(new
    FileWriter(booksFile, false))) {
```

```

        for (Book b: books.values()) {
            bw.write(b.toString());
            bw.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error saving books: " + e.getMessage());
    }
}

private void saveMembers() {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(
        "membersFile", false))) {
        for (Member m: members.values()) {
            bw.write(m.toString());
            bw.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error saving members: " +
            e.getMessage());
    }
}

```

```

public class Main {
    static LibraryManager lm = new LibraryManager();
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] a) {
        System.out.println("=== Library Management ===");

        while (true) {
            showMenu();
            switch (sc.nextLine().trim()) {
                case "1" -> addBook();
            }
        }
    }
}

```

case "2" → addMember();
 case "3" → actionBook("Issue");
 case "4" → actionBook("Return");
 case "5" → searchBooks();
 case "6" → sortBooks();
 case "7" → listMembers();
 case "8" → listBooks();
 case "9" → { System.out.println("Exiting -
 data saved. Bye!"); return; }
 default → System.out.println("Invalid choice,
 please try again");

static void showMenu() {
 System.out.println("An 1: Add Book 2: Add Member
 3: Issue Book 4: Return Book 5: Search");
 System.out.println("6: Sort 7: List Members
 8: List Books 9: Exit");
 System.out.println("Enter choice: ");

static void addBook() {
 System.out.print("Title: "); String t = sc.nextLine();
 System.out.print("Author: "); String a = sc.nextLine();
 System.out.print("Category: "); String c = sc.nextLine();
 Book b = lm.addBook(t, a, c);
 System.out.println("Added book:"); b.displayBookDetails();

static void addMember() {
 System.out.print("Name: "); String n = sc.nextLine();
 System.out.print("Email: "); String e = sc.nextLine();
 try {

```

Member m = lm.addMember(n, e);
System.out.println("Member added:");
m.displayDetails();
} catch (IllegalArgumentException ex) {
    System.out.println("Invalid email. Member not
    added");
}
}

```

```

static void actionBook(String type) {
    int bid = askInt("Book ID:");
    int mid = askInt("Member ID:");
    String result = type.equals("Issue") ? lm.issueBook(bid, mid)
    : lm.withdrawBook(bid, mid);
    System.out.println(result);
}

```

```

static void searchBooks() {
    System.out.println("Enter search title/auth or category:");
    var list = lm.searchBooks(sc.nextLine());
    if (list.isEmpty()) System.out.println("No books");
    else list.forEach(book::displayBookDetails);
}

```

```

static void sortBooks() {
    System.out.println("Sort by: 1. Title 2. Author
    3. category");
    System.out.println("choice:");
    String c = sc.nextLine().trim();
}

```

```

var list = switch (c) {
    case "2" -> lm.sortByAuthor();
    case "3" -> lm.sortByCategory();
    default -> lm.sortByTitle();
}

```

```

};

```

```
    list.forEach(Book::displayBookDetails);  
}
```

```
static void listMembers() {  
    System.out.println("Members:");  
    lm.allBooks().forEach(Member::displayMemberDetails);  
}
```

```
static void listBooks() {  
    System.out.println("Books:");  
    lm.allBooks().forEach(Book::displayBookDetails);  
}
```

```
static int askInt(String msg) {  
    while (true) {  
        System.out.print(msg);  
        try { return Integer.parseInt(sc.nextLine().trim()); }  
        catch (Exception ignored) { System.out.println("Enter a valid number"); }  
    }  
}
```

```
}
```